

Measuring a Synchronous API Contract

An empirical investigation of request lifetime, work duration, and client waiting time

Duration	Work mode	Code
75 minutes	Pairs	Provided

Central question: How does report-generation time affect the externally observable behavior of a synchronous report API?

Learning outcomes

- Measure API response time under controlled conditions.
- Distinguish a prediction, an observation, and a causal explanation.
- Identify the relationship between work duration and response time.
- Explain how the synchronous contract couples request lifetime and work lifetime.
- Derive a system requirement from measured evidence.

The current contract

A contract is an externally observable promise between components. It states what a client may expect from a server; it does not describe the server's internal implementation.

Current promise If the request succeeds, POST /v1/reports returns 200 OK with the completed report in the response body.

Because the completed report must be placed in the response, the handler must finish report generation before it can fulfill the contract. The client therefore waits while the work is performed.

Approximate relationship:

response time $\approx$ HTTP overhead + report-generation time

Experimental design

Research question

How does report-generation time affect the externally observable behavior of the synchronous report endpoint?

Hypothesis — complete before testing

If report-generation time increases, then _____ because _____.

Variables

Role	Variable
Independent	Artificial report-generation delay: 0 s, 3 s, 7 s, 12 s
Dependent	Response time, HTTP status, response bytes, complete report received, server completion log
Controlled	Endpoint, request body, consumer, reporting period, database contents, client command and server timeout

Before you measure: make predictions

Record predictions before running the server under each condition. Predictions are not graded for being correct; they are evidence of your reasoning before observation.

Delay	Predicted response time	Predicted status	Complete body?
0 s			
3 s			
7 s			
12 s			

At what point do you predict the current design will fail or become unreliable? Why?

Procedure

Part A — Start one experimental condition

Your instructor will provide the database connection value. Start the API with one artificial-delay condition:

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=3s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=7s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=12s
```

Controlled experiment Change only the report-delay value. Restart the server between conditions. Keep the request, data and measurement command unchanged.

Part B — Send and measure the request

In a second terminal, create request.json once:

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

Then run the same measurement command for every condition:

```
curl --silent --show-error \
  --output response.json \
  --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  http://localhost:4000/v1/reports
```

After every request, inspect the body:

```
cat response.json
```

Part C — Repeat consistently

1. Run the request three times for the current delay.
2. Record every result; do not discard unexpected results.
3. Inspect response.json and the server log after each request.
4. Stop and restart the server with the next delay.
5. Do not run different delay conditions simultaneously.

Raw measurements

Record the client-visible result and the server-visible result. A blank body, status 000, timeout or connection error is still an observation and must be recorded.

Condition: 0 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1					
2					
3					

Condition: 3 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1					
2					
3					

Condition: 7 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1					
2					
3					

Condition: 12 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1					
2					
3					

Summarize the evidence

For each condition, calculate the approximate mean response time:

$$\text{mean response time} = (T_1 + T_2 + T_3) \div 3$$

Delay	Mean response time	Successful responses	Complete reports
0 s		/ 3	/ 3
3 s		/ 3	/ 3
7 s		/ 3	/ 3
12 s		/ 3	/ 3

Observation

Write only what you measured. Do not explain the mechanism yet.

When report delay increased from _____ to _____, mean response time changed from approximately _____ to _____.

Explanation

Now explain the mechanism that produced the observations.

Why did response time change as the artificial delay increased?

What must the handler complete before it can send 200 OK with the report?

While the report is being generated, is the original request complete or outstanding?

Would asynchronous execution make the report faster, or would it change when the client receives acknowledgement?

Do not confuse cause with threshold A 10-second write timeout may expose the problem, but 10 seconds is not a universal boundary. Increasing the timeout moves the boundary; it does not remove the coupling.

From evidence to requirements

A requirement describes a property the system must provide. Do not write implementation mechanisms such as 202, worker, queue or polling in this section.

A valid report request should be acknowledged within _____.

Report generation should not depend on _____.

The client should be able to determine _____.

The system should preserve _____.

Report-generation duration should not determine _____.

The contract decision

The current contract promises completion in the original response. Based on your evidence and derived requirements, choose one option and justify it.

- ☐ Keep the current contract: return 200 OK with the completed report.
- ☐ Change the contract: acknowledge accepted work before the report is complete.

Justification:

In-class exit ticket

Submit one response per pair before leaving.

Element	Your response
Claim	The synchronous API is / is not suitable for potentially long-running reports because ...
Evidence	Provide at least two measured values.
Reasoning	Explain how the evidence follows from the current contract.
Requirement	State one requirement the current contract cannot reliably satisfy.

Complete outside class

Submit a short laboratory report containing the following sections:

- Research question and hypothesis
- Independent, dependent and controlled variables
- Raw measurements and calculated mean response times
- Observations stated without explanation
- Causal explanation of the observed behavior
- At least two limitations of the experiment
- Requirements derived from the evidence
- A proposed revised contract

Required reasoning chain

Layer	Question to answer
Evidence	What did we measure?
Problem	What coupling did the evidence expose?
Requirement	What property does the system now need?
Contract	What should the API promise the client?
Implementation	What mechanisms might provide that contract?

Important distinctions

Incorrect conclusion	Stronger conclusion
Async makes report generation faster.	Async can separate acknowledgement time from completion time; the work may take just as long.
The design fails after exactly 10 seconds.	The observed threshold depends on configuration; the underlying issue is lifetime coupling.
The timeout is the whole problem.	A larger timeout preserves the synchronous contract and merely permits longer waiting.
202 means the report succeeded.	202 means the server accepted the work; completion or failure must be observed later.

Next laboratory You will redesign the contract using Job + Worker + 202 Accepted + GET /jobs/{id}, then repeat the measurements and compare acknowledgement time with completion time.